Intégration Supabase et travail hors connexion

Guide de mise en place pour FONCIARCH

12 novembre 2026

Recommandation : utiliser Supabase pour la base centrale PostgreSQL, avec IndexedDB et une file de synchronisation sur chaque appareil.

Cette livraison ajoute la saisie hors connexion, la reprise des envois et le traitement explicite des conflits. Les coordonnées du projet sont configurées localement. L’accès distant et le schéma Supabase restent à vérifier.

Le dossier technique joint contient la migration SQL complète, les variables à renseigner, les scripts de transfert et les contrôles de recette.

L’authentification reste suspendue à votre demande. Réactiver un contrôle d’accès avant une exposition publique du serveur.

1 Backend Supabase

Supabase est le seul backend de FONCIARCH. PostgreSQL conserve les relations entre titres, boxes, catégories et mouvements, les contraintes de doublons et l’historique transactionnel. Aucun autre moteur serveur n’est sélectionnable dans l’application.

Le fonctionnement hors connexion repose sur IndexedDB et une file d’opérations sur l’appareil. Cette copie locale permet de travailler sans réseau ; elle est synchronisée exclusivement avec Supabase par le serveur Next.js. Elle ne constitue pas un second backend.

La résolution des conflits est explicite : une version périmée d’un titre ne remplace pas silencieusement une modification plus récente reçue par le serveur. L’absence de réseau ou une erreur Supabase conserve les saisies dans la file locale, sans bascule vers une autre base.

Référence : PostgreSQL dans Supabase (https://supabase.com/docs/guides/getting-started/the-supabase-way-to-build).

2 Ce qui est livré

Les icônes de l’onglet, de l’application installée et de l’écran d’accueil proviennent du logo FONCIARCH. Le manifeste ouvre l’application dans une fenêtre dédiée lorsque le navigateur le permet. Le service worker conserve l’interface, ses scripts, ses styles et ses polices en production.

Le registre chargé, les boxes, les paramètres et les opérations en attente sont conservés dans IndexedDB. La création, la correction, les changements de statut, les déplacements motivés et l’import CSV peuvent être enregistrés sans serveur accessible. La recherche et les rapports utilisent la copie locale disponible.

Chaque saisie est enregistrée localement avant l’affichage d’un succès. L’envoi automatique se déclenche à l’ouverture, au retour du réseau, lorsque l’application redevient visible et périodiquement lorsqu’elle reste ouverte. Un identifiant unique et un reçu serveur permettent de rejouer une requête après une coupure sans créer une seconde fois le titre ou le box.

L’en-tête affiche un état discret. Les opérations en attente et les conflits se consultent dans Paramètres > Données et synchronisation. Les messages de réussite sont intégrés à la page ; il n’y a pas de notification publicitaire, de demande d’autorisation de notification ni de fenêtre d’alerte automatique.

L’adaptateur Supabase, la migration SQL et les outils de transfert sont fournis. Les clés du projet sont configurées uniquement dans le fichier local protégé ; aucun secret ne figure dans ce guide. La vérification distante a été empêchée par une erreur DNS dans cet environnement : l’accès cloud et la présence du schéma doivent encore être confirmés.

3 Architecture et limites à connaître

Le parcours est le suivant : interface React > enregistrement IndexedDB > file locale > API Next.js > fonctions PostgreSQL Supabase > reçu > suppression de l’opération confirmée dans la file locale.

Le service worker gère les fichiers de l’application. Il ne met pas les réponses de l’API en cache HTTP. IndexedDB constitue une copie distincte des données métier, accessible à l’application sous la même adresse web. Les données locales ne sont pas chiffrées par cette version ; un appareil partagé doit être administré et protégé par le service.

Le travail hors ligne après fermeture et réouverture nécessite une première préparation complète en ligne : ouverture de l’application, chargement du registre et activation du service worker. Un appareil qui n’a jamais chargé le site ne peut pas télécharger l’application sans réseau.

La fermeture de l’application suspend l’envoi. Les saisies reprennent leur synchronisation à la réouverture avec une connexion disponible. Il n’est pas promis de synchronisation lorsque le navigateur est fermé. La simple présence du Wi-Fi ne garantit pas l’accès au serveur : en cas d’échec, la file est conservée et une nouvelle tentative est effectuée.

Les données dépendent de l’adresse du site et du navigateur. Passer de localhost à un domaine, changer de profil, utiliser la navigation privée ou effacer les données du site crée ou supprime une copie différente. Ne jamais effacer les données du navigateur tant que des saisies sont en attente. Le bouton Exporter ma copie locale fournit une sauvegarde JSON. Une demande de stockage persistant est également disponible ; le navigateur peut la refuser.

La version actuelle charge un instantané complet du registre. La pagination serveur, les téléchargements incrémentaux, les suppressions et la montée en charge vers plusieurs centaines de milliers de titres restent à concevoir et à mesurer. La première version concerne un seul service et un seul cadastre. La règle de série reste indicative et doit être confirmée sur le terrain.

4 Préparer le projet Supabase

1. Créer un compte Supabase, puis une organisation et un projet pour FONCIARCH. Conserver séparément un projet de test et un projet de production.

2. Choisir la région en tenant compte de la latence observée depuis le service, des obligations de conservation des archives et de l’offre disponible. Cette décision doit être confirmée par le responsable du service.

3. Définir un mot de passe de base robuste, le conserver dans un gestionnaire de secrets et attendre la disponibilité du projet.

4. Relever l’URL du projet et une clé secrète serveur dans les réglages des clés API. Ne pas copier cette clé dans un message, une capture, le code source ou une variable commençant par NEXT_PUBLIC.

5. Vérifier les quotas du plan choisi, les sauvegardes proposées, la rétention et la possibilité de restauration. Ne pas considérer une copie du navigateur comme une sauvegarde de la base centrale.

L’adaptateur fourni utilise l’API REST de Supabase depuis Next.js et n’exige pas l’ajout d’un SDK dans le navigateur. La clé publique n’est pas nécessaire pour ce mode d’accès. La clé secrète autorise des opérations privilégiées et doit rester exclusivement sur le serveur.

Référence : clés API Supabase (https://supabase.com/docs/guides/getting-started/api-keys).

5 Installer les tables et fonctions

Dans le SQL Editor du projet de test, ouvrir et exécuter le fichier complet :

```
supabase/migrations/202611120001_fonciarch.sql
```

Le fichier contient les tables, contraintes, index et fonctions nécessaires. Il ne crée aucun titre, box ou utilisateur fictif. La table d'identité technique crée uniquement l'identifiant de la base utilisé pour détecter un changement de source.

Les tables sont : fonciarch_identity pour l'identité de la source ; fonciarch_settings pour le service ; fonciarch_boxes pour les emplacements ; fonciarch_titles pour les titres ; fonciarch_activities pour le journal ; fonciarch_operations pour les reçus de synchronisation.

Les fonctions exposées au serveur sont fonciarch_workspace pour lire le registre, fonciarch_apply_operation pour appliquer une opération une seule fois et fonciarch_restore pour restaurer une copie dans un registre vide. Les fonctions auxiliaires construisent les fiches et appliquent les contrôles métier dans la même transaction.

Les numéros des titres sont stockés en texte pour conserver les zéros initiaux. La paire numéro et catégorie est unique dans cette première version à cadastre unique. La relation entre titre et box interdit un emplacement inexistant. L'import s'exécute dans une transaction : une erreur empêche la validation de tout le lot.

Vérifier les tables et la restriction des clés publiques :

```
select tablename, rowsecurity from pg_tables
where schemaname = 'public'
and tablename like 'fonciarch_%';

select has_table_privilege(
'anon', 'public.fonciarch_titles', 'SELECT'
) as lecture_publique;

select has_function_privilege(
'anon',
'public.fonciarch_apply_operation(uuid,text,jsonb)',
'EXECUTE'
) as execution_publique;
```

Les six tables doivent avoir RLS activé. Les deux privilèges publics doivent être faux. Ne pas ajouter de politique ouverte telle que USING (true). Le test SQL du dossier supabase/tests est destiné à une base de test vide et annule ses enregistrements de test ; ne pas le lancer sur une base de production.

6 Configurer le serveur Next js

Le projet nécessite Node.js 24 ou une version compatible ultérieure. Copier .env.example vers .env.local sur l'environnement de travail puis renseigner :

```
SUPABASE_URL=https://VOTRE_PROJET.supabase.co
SUPABASE_SECRET_KEY=VOTRE_CLE_SECRETE_SERVEUR
```

Les valeurs en majuscules sont des emplacements à remplacer, pas des identifiants utilisables. Ne pas modifier le nom de SUPABASE_SECRET_KEY en NEXT_PUBLIC_SUPABASE_SECRET_KEY. Conserver .env.local hors du dépôt. Sur l'hébergement, déclarer les mêmes variables dans le gestionnaire de secrets du serveur, puis redémarrer ou redéployer.

L'API utilise uniquement Supabase via src/lib/backend.ts. Il n'existe plus de sélecteur de backend ni de base serveur de secours. Si la configuration ou la migration est manquante, si la clé est refusée ou si le serveur est inaccessible, les opérations locales restent en attente.

Une clé sb_publishable ou une ancienne clé de rôle anon ne peut pas remplacer SUPABASE_SECRET_KEY. Après configuration, lancer npm run supabase:check pour tester uniquement la RPC de lecture, sans afficher de clé ni de fiche du registre.

Pour préparer et lancer la PWA de production :

```
npm install
npm run typecheck
npm test
npm run build
npm start
```

Le script build crée public/pwa-assets.json et lie la version du cache aux ressources produites. Conserver le dossier public avec le déploiement. Un simple lancement de Next sans le script de préparation ne constitue pas la procédure de build de la PWA. Le mode npm run dev désactive volontairement le service worker de production pour éviter de servir des anciens fichiers pendant le développement.

Ouvrir le site sous HTTPS, ou sur localhost pour les essais sur le même ordinateur. Une adresse HTTP du réseau local ouverte depuis un téléphone n'offre pas les mêmes conditions de fonctionnement qu'un site HTTPS. Dans Paramètres > Application FONCIARCH, vérifier que l'application est disponible hors connexion avant de couper le réseau.

Référence : installation des PWA (https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable).

7 Sauvegarder et restaurer un registre Supabase

Pour un nouveau service, renseigner le nom, le cadastre et la localisation dans Paramètres après l’installation du schéma. Créer ensuite les boxes, puis les titres ou l’import CSV. Les saisies peuvent être enregistrées sur l’appareil pendant une indisponibilité du serveur, mais ne sont centrales qu’après synchronisation.

Pour transférer un registre entre environnements Supabase, suspendre la saisie pendant le transfert. Sur chaque appareil, ouvrir l’application et vérifier qu’il ne reste aucune opération en attente ou en conflit. Exporter une copie locale par précaution. Les saisies d’un appareil éteint ou hors ligne ne sont pas automatiquement incluses dans la copie du serveur.

Avec le serveur de l’environnement source en fonctionnement, exporter l’instantané réel :

```
node scripts/export-server.mjs \
http://localhost:3000 \
output/fonciarch-avant-transfert.json
```

Le script refuse d’écraser un fichier existant. Protéger cette copie car elle contient les archives et le journal du service.

Après installation de la migration SQL sur un projet Supabase cible vide, renseigner les variables du projet cible dans .env.local et exécuter :

```
node --env-file=.env.local \
scripts/restore-supabase.mjs \
output/fonciarch-avant-transfert.json
```

La restauration est transactionnelle et refuse un registre cloud déjà utilisé. Elle conserve les identifiants, notes, statuts, dates et événements de l’instantané. L’identité de la source reste propre au projet Supabase cible. Aucun ancien compte d’authentification local n’est transféré.

Contrôler les nombres de titres et de boxes ainsi que plusieurs fiches, puis redémarrer avec les coordonnées du projet cible. Conserver les sauvegardes jusqu’à validation de la recette. Un appareil qui garde des écritures pour une autre source bloque leur envoi : exporter et traiter ces écritures avant de les écarter. Ne pas effacer IndexedDB pour contourner ce contrôle.

8 Utiliser le mode hors connexion

Préparer une fois chaque appareil en ligne. Ouvrir FONCIARCH, attendre la fin du chargement et vérifier les fiches utiles. Installer l’application si le navigateur propose cette possibilité. Dans Paramètres, vérifier l’état de l’application hors connexion et demander le stockage persistant si nécessaire.

Une fois le réseau coupé, consulter les titres déjà chargés, créer un box si nécessaire, ajouter un titre, corriger une fiche, choisir un statut ou enregistrer un déplacement avec un motif. Le titre devient immédiatement visible dans la copie locale. L’en-tête affiche le nombre d’opérations en attente. Les exports utilisent les données visibles sur cet appareil.

La saisie peut continuer sans fenêtre d’alerte automatique. Un succès signifie que l’écriture locale est terminée ; il ne signifie pas encore qu’un autre appareil l’a reçue. Les opérations non confirmées sont visibles dans Paramètres > Données et synchronisation.

À la reconnexion, laisser l’application ouverte. La file est envoyée dans l’ordre, avec les dépendances nécessaires. Un box précède les titres qui l’utilisent. La correction d’un titre créé hors ligne attend sa création serveur, puis utilise sa version confirmée. Le navigateur retire une opération de la file uniquement après réception de son reçu.

Si le serveur a validé une opération mais que sa réponse a été perdue, le même identifiant est renvoyé. Le serveur rend le reçu précédent sans créer un nouvel enregistrement ni répéter le mouvement. Une panne réseau ou serveur conserve la file. Une erreur de validation ou un conflit bloque l’envoi à examiner ; aucune écriture n’est effacée automatiquement pour faire disparaître l’erreur.

9 Examiner un conflit

Exemple : deux appareils ont chargé la même fiche, puis chacun modifie cette fiche hors ligne. Le premier envoi est validé. Le second est refusé si sa version de départ est devenue périmée. Sa saisie reste consultable localement.

Dans Paramètres > Données et synchronisation, ouvrir Examiner et résoudre. Comparer les valeurs du serveur à la saisie locale. Pour une modification ou un déplacement portant sur le même identifiant, le choix Choisir ma version prépare un nouvel envoi à partir de la version serveur affichée. Une nouvelle modification concurrente sera contrôlée à nouveau.

Le choix Écarter la saisie locale retire l’opération et ses opérations dépendantes après confirmation intégrée dans la page. Les données déjà reçues par le serveur ne sont pas supprimées. Exporter la copie locale avant ce choix si des informations doivent être récupérées.

Un doublon lors d’une création ne doit pas être transformé silencieusement en modification d’un titre existant. Vérifier le numéro et la catégorie. Conserver l’export, écarter la création rejetée si elle est erronée, puis saisir la correction avec le bon numéro ou ouvrir le titre déjà présent. Un box inconnu doit être créé ou corrigé avant un nouvel import.

La file ne doit pas être manipulée directement dans les outils du navigateur. Le journal conserve les opérations reçues par le serveur ; l’identité de l’agent n’est pas établie tant que l’authentification reste suspendue.

10 Sécuriser le passage en production

L’authentification a été retirée de l’application à votre demande. Dans l’état actuel, une personne qui atteint le serveur Next.js peut consulter et modifier le registre. Le fait que les tables Supabase refusent les clés publiques ne protège pas une API Next.js volontairement ouverte et utilisant une clé secrète.

Avant de rendre l’application accessible sur Internet, prévoir Supabase Auth pour les agents, valider les sessions et les rôles côté serveur et lier chaque opération à l’utilisateur identifié. Définir des profils et des droits par cadastre. Si un accès direct depuis le navigateur est ajouté, écrire et tester les politiques RLS correspondantes. La clé secrète ne doit jamais servir de mécanisme d’autorisation utilisateur.

La connexion initiale et le renouvellement d’une session nécessitent le réseau. En fonctionnement hors ligne, les données autorisées et les écritures en attente doivent être liées au bon utilisateur et à son appareil ; un changement de compte ne doit jamais réexpédier la file d’un autre agent. Cette séparation n’est pas implémentée dans le mode temporaire sans authentification et doit précéder l’usage partagé en production.

Prévoir HTTPS, une politique d’appareils maîtrisés, la gestion des sauvegardes et un test de restauration. Définir qui peut exporter les archives et comment conserver les copies JSON. Vérifier l’offre Supabase retenue pour les sauvegardes et la restauration ; aucune sauvegarde cloud ni restauration réelle du projet n’a encore été effectuée par cette intervention.

Références : Row Level Security (https://supabase.com/docs/guides/database/postgres/row-level-security), clés et privilèges serveur (https://supabase.com/docs/guides/getting-started/api-keys).

11 Recette avant utilisation courante

Effectuer les essais dans un projet de test et sur les navigateurs réellement utilisés par les agents.

1. Ouvrir un registre vide, renseigner le service et créer deux boxes. Vérifier leur présence dans Supabase.

2. Ajouter un titre avec un numéro commençant par zéro. Recharger la page et vérifier numéro, catégorie, box et statut.

3. Terminer la préparation PWA en production. Couper le réseau et recharger l’application : l’interface et les données chargées doivent s’ouvrir.

4. Sans réseau, créer un titre, le corriger puis le déplacer. Fermer et rouvrir l’application. Vérifier la copie locale et les opérations en attente.

5. Rétablir le réseau et laisser l’application ouverte. Vérifier que la file se vide et que le titre apparaît une seule fois depuis un second appareil.

6. Simuler une coupure après l’écriture serveur, puis renvoyer la même opération. Vérifier qu’il n’existe qu’un seul titre et un seul événement pour cette opération.

7. Modifier la même fiche depuis deux appareils. Vérifier le conflit et les deux choix de résolution, sans écrasement silencieux.

8. Importer un CSV avec une erreur et un doublon. Vérifier le bilan et l’absence d’écriture partielle du lot rejeté côté serveur.

9. Simuler un serveur indisponible, un stockage local plein et un changement de source avec une file non vide. Vérifier la conservation des saisies et les messages intégrés à la page.

10. Tester la sauvegarde, la restauration, les règles d’accès et les appareils partagés avant l’ouverture du service à plusieurs agents.

Les tests automatisés du dépôt couvrent le moteur de synchronisation, l’API avec des réponses Supabase simulées et la logique du service worker. Le schéma et les fonctions ont été exécutés sur PostgreSQL 18 isolé. Cela ne remplace pas les essais d’installation et de stockage dans de vrais navigateurs, ni la recette d’un projet Supabase avec ses propres clés. Les navigateurs, le quota réel et le déploiement cloud restent à vérifier sur votre environnement.

12 Fichiers fournis et ordre de mise en place

- .env.example : noms des variables et emplacements à remplacer.

- supabase/migrations/202611120001_fonciarch.sql : schéma, contraintes, fonctions, restrictions d’accès et restauration contrôlée.

- src/lib/backend.ts : accès REST Supabase exclusif côté serveur, sans repli vers un autre moteur.

- src/lib/offline : modèle local, stockage IndexedDB, projection des opérations, synchronisation et reçus.

- src/components/offline-workspace.tsx : état de synchronisation et traitement des opérations dans l’interface.

- public/sw.js et scripts/build-pwa-assets.mjs : cache versionné de l’application et préparation des ressources.

- scripts/export-server.mjs et scripts/restore-supabase.mjs : transfert contrôlé d’un registre existant.

- scripts/test-workspace.mjs, scripts/test-offline.mjs et scripts/test-pwa.mjs : vérifications reproductibles.

- supabase/tests/validation.sql : contrôles SQL sur une base de test vide.

Ordre conseillé : préparer le projet de test ; exécuter le SQL ; configurer les secrets serveur ; transférer ou démarrer vide ; construire la version PWA ; faire la recette en ligne et hors ligne ; rétablir l’authentification avant une exposition publique ; préparer les sauvegardes et la mise en production.

La version Markdown et le dossier technique contiennent le code complet à utiliser. Les fichiers de configuration ne contiennent aucune vraie clé. Ne publiez pas les exports du registre avec les fichiers du site.